SAMS Kernel

What has run on real hardware, and how to try the rest

Version 1.35.1 · 28 October 2026
For a TI-99/4A with an F18A video card and a SAMS memory card
MIT License, © 2026 Rebecca Sutter

Contents

Run on real hardware before (by Press and The Lantern Coast, the kernel's ancestors)

The walk

js99er's TIPI emulation and a frozen machine

What each unverified claim rests on

The disk start (1.25.0; nothing of it has run on a real machine)

RUN PROGRAM FILE (1.31.0; nothing of it has run on a real machine)

Module programs (1.32.0; nothing of it has run on a real machine)

Scrolling (1.35.0, with the sprite and aiming demos of 1.33-1.34; none of it has run on a real machine)

How to report

Everything in this project is verified in a simulator (a TMS9900, the F18A, SAMS, a keyboard, joysticks, TI's own console and disk controller ROMs, a model of the sound chip). That finds almost every programming mistake and none of the hardware's surprises. This list is the honest boundary, in the order I would walk it on a real machine or in js99er (F18A and SAMS on). Each step says what to do, what you should see, and what to tell me if you do not.

Run on real hardware before (by Press and The Lantern Coast, the kernel's ancestors)

SAMS paging with a 1M-or-smaller card, the keyboard decode, the F18A 80-column text screen and colour roles, files through TI's disk controller, the FinalGROM "8" image format and bank switching. **Not** run on real hardware in this form: the loader as rewritten, the module system, the page pool, and everything below. **Reports about Press (31 October 2026, passed on to the project owner by users; their names were not recorded):** Press runs well on real hardware ("the command keys feel much more natural on the original keyboard"), and runs on the latest Classic99 with the PICO9918 support enabled, where CTRL-Q works. These are reports about Press, the ancestor, not about this kernel's loader, module system or page pool. js99er with a 16M SAMS has run the kernel (which found the QUIT bug in 1.3.0, fixed in 1.3.1). **31 October 2026:** the project owner reports that the workbench and every example of release 1.15.0 run correctly in js99er, with its F18A setting and with its PICO9918 setting (the SAMS size and the disk setting were not recorded; 1.15.0 has `LINEIN` and resident code but not `pool_max_pages`, which came in 1.16.0). That is an emulator, not a TI: it does not show real timing, a real F18A or PICO9918, a real SAMS card or a real FinalGROM.

Routes. Steps 1-8 run prebuilt images (`images/`), which need a **[PC]** only to have been made: copy them to the cartridge and the disk and you can do them with no PC. Step 9 has two forms below: **[PC]** builds `tests/apps/files2`; **[TI]** runs `COPY/A99` from the workbench disk (the same test, two files open at once). Steps 10 and 11 are **[TI]** by nature (the workbench itself).

The walk

#	Do	You should see	If not, tell me
1	Load `images/hello8.bin`. Type. FCTN =	a blue screen, a title bar, your typing echoed; FCTN = returns to the console's title screen with the display normal	which message (a refusal screen names its reason), a blank screen, or garbled colours
2	`images/skdiag8.bin`, CTRL T	eight rows, all **PASS**; the first shows the page count (128 for 512K, 256 for 1M, 512 / 1024 / ... / 4096 for bigger cards); a disk in drive 1 for the last row	which row failed; the step and error on the files row
3	Leave diag with FCTN = (**with a card above 1M if you have one**)	the console title screen; the card's mapping must be off	a hang or blank screen at the reset
4	In diag, press keys and chords	each shows its code and modifiers; CTRL 3 cycles the colour schemes	a chord with the wrong code (name it)
5	`images/videomodes8.bin`, keys `1-9 0 A-E`; FCTN =	fifteen screens, one for each display mode and option; each matches `docs/VIDEO-MODES.md`	which mode, and what it looked like; PICO9918 or F18A, and its firmware
6	`images/music8.bin: 1`, SPACE, SPACE, `S, 2, B, W`	the tune (three voices and a beat, about 16 s, looping), silent while paused and then the same note, an A (440 Hz), a beep, and the tune playing through a two-second "work" loop	the tune is too fast or slow, a wrong note, a hiss or click, or the noise voice too loud (a PAL console needs `"fps": 50`)
7	`images/joystick8.bin`, each joystick in each direction, and fire	the matching label goes bold; the fire count rises once a press; `1`, `2`, `3` make	the wrong direction or

		joystick 1, 2 or both type keys (the "Keys typed" count and the last code change; UP, DOWN, LEFT, RIGHT, ENTER are `C5 D8 D3 C4 0D`)	joystick, a key that sticks, ghosting with a key held
8	`images/diskbrowser8.bin` with a disk in DSK1	the catalog; ENTER on a text file shows it; scroll; FCTN 9 back	a hang, a wrong size or type, a garbled name
9	Several files, **[TI]**: in the workbench (the disk has a sample `SRC`), N `DSK1.COPY/A99` ENTER, A, R, ENTER; the disk then holds `DST`. **[PC]**: build `tests/apps/files2` (`python3 tools/skbuild.py tests/apps/files2`) and make a disk from two text files named `src` and `src2` (`python3 tools/skdisk.py d.dsk src src2`), then load `build/files2/files28.bin` with that disk in DSK1	nothing on the screen; afterwards the disk holds `DST`, a copy of `SRC` (check it in your usual disk manager)	the disk controller or TIPI in use; whether `DST` is complete
10	`images/workbench8.bin` with `images/workbench.dsk` in DSK1: N `DSK1.HELLO/A99` ENTER, A, then R	"Assembled: 194 bytes at >C000", then the hello program; **time the assembly with a watch and tell me** (the simulator ran 1.9 million instructions); then restart (hello has no way back) and `BEEP/A99`: A, R, keys 1-8 play a scale, FCTN 9 returns to the workbench	the time; any error list; a hang
11	In the workbench: E on `HELLO/A99`, type a line, CTRL S, CTRL A	the edit saves and re-assembles; a mistyped mnemonic is reported with its line	an editor key that does the wrong thing
12	Cards above 1M, if you have one: step 2 again, then 6 and 10	the page count is right and everything else as before	the first thing that differs
12a	In the workbench: N `DSK1.ASK/A99` ENTER, A, R; type `ann` ENTER, `bob` ENTER, then FCTN E	`Hello, ANN`, `Hello, BOB`; FCTN E puts BOB back in the field; the arrows, FCTN 1, CTRL H and FCTN 2 edit; FCTN 9 returns	a key that does the wrong thing (name it), a field that does not redraw cleanly

TIPI steps (if you have one; each is **[TI]**):

#	Do	You should see	If not, tell me
13	`images/diskbrowser8.bin` with `DSK1.` mapped to a directory on the TIPI	the directory's catalog; a text file opens	whether the catalog request is answered at all
14	the workbench on the TIPI: a source in the mapped directory (or `PI.` paths), N, A, R	assembles and runs, as from a disk; includes found	the first failing name, and its error
15	`COPY/A99` with `SRC` on `DSK1.` (TIPI) and, if you also have a TI disk controller, the copy going between them	`DST` appears; with both cards present nothing hangs, whichever device is used first	which direction fails
16	`CLOCK/A99` from the workbench disk, ENTER	today's date and the time	"no clock" with a TIPI attached, or a wrong date

js99er's TIPI emulation and a frozen machine

js99er's TIPI setting has three values (read from its source, `tipi.ts`): **No TIPI emulation**; **Mouse** (a TIPI at CRU `>1200`, no network); **Full** (a TIPI at `>1000` whose Pi is a **websocket server**, by default `ws://localhost:9901/tipi`). In Full, js99er suspends the whole CPU whenever that websocket is closed or cannot be opened, and retries every two seconds: with nothing listening the machine freezes at start-up, before the console's menu, whatever cartridge is loaded (it was reported once for this kernel, on 31 October 2026, before the menu appeared, so it is not this software). The TIPI's own power-up pulses its reset line, which makes the proxy close the connection, so a freeze of a couple of seconds at each reset is also normal. If a machine freezes with Full emulation: set the setting to No TIPI emulation (it should run), check that the websocket server is reachable at the URI in js99er's options, and look in the browser's developer console for "TIPI websocket opened" or "closed". The TXA project reports (1 November 2026, tested on js99er): TI's own controller

reads single-density images only, and js99er's "Generic Disk" setting reads double-density images that its TI-controller setting misreads, which is the setting for `workbench-360k.dsk` and `sources-360k.dsk`. The owner also reports (31 October 2026) that Press, the kernel's ancestor, works with a TIPI **on real hardware** (so a real TIPI and its Pi answer Press's file calls; the TIPI hardware version was not recorded): that supports the file-call path this kernel inherited from Press, not what this kernel changed (the power-up of every card, up to 1.16.0, and the search across several cards). Mouse needs no server: **the project owner reports (31 October 2026) that the workbench runs correctly in js99er with Mouse emulation**, which puts the real TIPI DSR in the machine at `>1200` (the kernel version was not recorded; the latest delivered was 1.16.1). That shows the real TIPI DSR does not disturb the kernel; it does not show files on a TIPI, which need a Pi behind a Full-emulation websocket. The websocket URI's default is `ws://localhost:9901/tipi`; the owner's setting was thought to be a different port with nothing listening, which would freeze Full emulation as described.

What each unverified claim rests on

Claim	Rests on	Where it could be wrong
a 12-bit SAMS register is the page with its bytes swapped (even byte the low 8 bits)	js99er's `ams.js`	a real card above 1M might lay the odd byte out differently
joysticks are keyboard columns 6 and 7, rows fire / left / right / down / up	my knowledge of the console's scan, and the simulator's model of it	the row order, or a joystick that needs a different column select
three files at once through the controller's three buffers; the controller finds a file by its PAB's address	TI's own disk controller ROM, running in the simulator (it did so correctly)	TIPI, CF7+ and other devices may differ
the music player writes the right bytes at the right frames	tests against a model of the sound chip	how it sounds; the noise model is an approximation
the workbench's assembler is right	byte-identical to xas99 for every example and two probe sources	an instruction or directive the examples do not use
TIPI: the device names, the clock record, the power-up of every card, includes in the main file's directory	the TIPI wiki as Press's design used it (Press's file speed was measured on a TIPI); a model of a TIPI in the simulator	a real TIPI's DSR, its open-file limit and its catalog
the page tests at power-up take about a second on a 16M card	0.25 million instructions in the simulator	an estimate; I have not timed it

The disk start (1.25.0; nothing of it has run on a real machine)

Build with `python3 tools/skbuild.py examples/hello --disk --geometry sssd` and put `build/hello/hello.dsk` on a real disk (or use a CF7+, TIPI or other volume). On a TI-99/4A with an F18A (or PICO9918), a SAMS card and a disk controller:

Step	Do	Good looks like	What it settles
D1	Editor/Assembler, option 5, `DSK1.HELLO`	the drive runs, then the application's screen (`Hello, SAMS Kernel`); a refusal screen names what is missing	that option 5 starts the bootstrap at `>C000` and that the DSR LOAD of the bank files works on your controller
D2	the same with the SAMS card left mapped by an earlier program (run the workbench cartridge, quit, then D1)	the same	the mapper is reset by the bootstrap
D3	a program with `"prun": true` (`tests/apps/prun_a`, `prun_b` on one disk): run ALPHA, press G	BETA's screen	`PRUN` and the handoff block
D4	D3 with the programs on `DSK2.` and ALPHA started from `DSK2.`	the same	the device probing and the handoff's device
D5	note the time from the return key to the application's screen	a number	how long a disk start takes on a floppy (not known)
D6	the workbench from disk;	the menu, the catalog `Assembled: 104 bytes at >C000`	the disk started

D6	the workbench from disk: `images/workbench-diskstart-360k.dsk` (360K: a double-density controller, a CF7+ or a TIPI), option 5, `DSK1.WBENCH`; then `C`, pick `HELLO/A99`, `A`, `R`	the menu, the catalog, `Assembled: 194 bytes at >C000`, the program	the disk-started workbench reads and assembles from its own disk
D7	after D1 (Hello), press FCTN = (js99er: Alt =)	the console's title screen, **24 rows, nothing below the copyright line** (1.35.1: before it the title came up in 30 rows with the colour table drawn as garbage on its bottom row, because the disk start lost the F18A flag the exit's restore depends on)	that quitting from a disk-started program puts the F18A back as from a cartridge's

RUN PROGRAM FILE (1.31.0; nothing of it has run on a real machine)

With `workbench8.bin` (or any cartridge built with `"ea5": true`) on the console's menu, choose **RUN PROGRAM FILE** (the second program):

Step	Do	Good looks like	What it settles
R1	the prompt: a title, `FILE NAME?`, `DSK1.` on the name line and a **solid blinking block** after it; type a name, `FCTN S` to delete one character	every key typed gives one character (also when keys overlap), none doubled, none lost; `FCTN S` takes off exactly one; a held key gives one character, not a run	that the console's screen and the matrix scan read the keys on your machine, and that contact bounce is handled (1.32.1: reported doubled and lost keys and a backspace that cleared the name)
R2	a program you know runs under the Editor/Assembler's option 5 (`DSK1.UTIL1` style files): type its name	it runs as under the module	that a real option 5 image runs with the console as the runner leaves it (interrupts on, GPL workspace); **tell me which program and what it does**
R3	`images/hello-diskstart-sssd.dsk` (the hello program as a kernel disk program): `DSK1.HELLO`	the kernel's hello screen	a kernel disk program through the runner
R4	a name that is not on the disk; `FCTN =`	`I/O ERROR 7`; the console restarts	the error path and the quit

Module programs (1.32.0; nothing of it has run on a real machine)

With `workbench8.bin` on the console's menu, **RUN PROGRAM FILE**, and `images/modprog-demo.dsk` in a drive:

Step	Do	Good looks like	What it settles
M1	`DSK1.GUEST`	the machine checks, then the kernel's screen with `Module program on disk`	that a module program reaches the kernel through the runner and the loader, on real SAMS and F18A; that the module's bytes survive in VRAM through the loader's checks
M2	FCTN = , then RUN PROGRAM FILE again	the console resets; the prompt comes back; M1 works a second time	that nothing of the first run is left in a way that stops the second
M3	run a program built for another build (one built against a `kernel.equ` of another release, or `skbuild.py` of an app with `"modprog": true` and different options)	`FOR ANOTHER KERNEL BUILD` with two IDs, and nothing starts	the refusal on a real console

Scrolling (1.35.0, with the sprite and aiming demos of 1.33-1.34; none of it has run on a real machine)

`images/scroll8.bin` as a cartridge (js99er: the file is named `...8.bin`; or FinalGROM) on a console with an F18A:

Step	Do	Good looks like	What it settles
S1	start it and watch for a minute (it walks right	a coloured world (blue water with waves, sand, green grass, dark-green forest, grey rock, brown roads, red-roofed houses) sliding	that the F18A's two-page layer 2 and the base swap

	along the top edge, down, left, up)	smoothly, with a yellow figure at the middle that does not move; **no seam, tear or jump** as the view passes X = 256 and 512 (a third of the way, two thirds) and Y = 192	work as the renderer (js99er's rules) says: the thing most worth looking at
S2	the arrow keys (FCTN S D E X), and joystick 1 if you have one	the view moves 8 pixels a press (2 a frame with the joystick), stops at the world's edges, and the walk starts again 2 seconds after the last press	the input path, and the clamping at the edges
S3	note any tile that is wrong (a row or column of the wrong tile at the screen's right or bottom edge while moving)	none	that the strips written as the window moves arrive in time, with the video chip's own timing, which the simulator does not have
A1	`images/aim8.bin`: the turret in the middle fires every second or so at the ring	the ring drifts and bounces in the box; a bullet through its hollow middle is a miss; SHOTS and HITS on the top line	pixel-exact collision (`SPMPIX`), `FXATN2` and `FXPOL` on a real F18A's sprites
A2	`images/spritemotion8.bin`	eight balls bouncing off the walls and each other	the motion table, `SPMBOX`, `SPMHIT` and `SPMSWP` on real sprites

How to report

A short note per step is enough: the step number, what you saw, the machine (console, F18A or PICO9918 with its firmware, SAMS size and brand, disk controller or TIPI), and for emulators the version and settings. A photograph of a refusal screen or the self-test is the best evidence there is.